



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS IV: Redes, Comunicaciones e Internet

**Tema 124. Aplicaciones móviles. Características, tecnologías,
distribución y tendencias**

AUTOR: Alba Cabarcos Bermúdez

Actualización 2017

ÍNDICE

1	Introducción a las aplicaciones móviles	3
2	Características y tecnologías	4
2.1	Aplicaciones web móviles	4
2.2	Aplicaciones nativas	6
2.2.1	Aplicaciones Android	7
2.2.1.1	Arquitectura de la plataforma Android.....	8
2.2.1.2	Componentes	9
2.2.1.3	Entornos de desarrollo	11
2.2.2	Aplicaciones iOS	11
2.2.2.1	Arquitectura de la plataforma iOS	11
2.2.2.2	Objetos más importantes en una aplicación iOS	12
2.2.2.3	Entornos de desarrollo	13
2.2.3	Aplicaciones Windows	14
2.2.3.1	La Plataforma Universal de Windows (UWP: Universal Windows Platform)	14
2.2.3.2	Entornos de desarrollo	15
2.2.4	Desarrollo nativo multiplataforma	16
2.2.4.1	Arquitectura de desarrollo con Xamarin.....	17
2.2.4.2	Entornos de desarrollo	19
2.3	Aplicaciones híbridas	19
2.3.1	Frameworks para el desarrollo de aplicaciones híbridas	20
2.4	Conclusiones	21
3	Distribución de aplicaciones móviles.....	22
3.1	Distribución de aplicaciones Android	22
3.2	Distribución de aplicaciones iOS.....	23
3.3	Distribución de aplicaciones Windows	23
3.4	Conclusiones	24
4	Tendencias	25
4.1	Ventas de teléfonos y sistemas operativos móviles más utilizados	25
4.2	Aproximaciones al desarrollo móvil más utilizadas	26
4.3	El paradigma Mobile First	27
4.4	Android Things.....	27
4.5	Conclusiones	28
5	Conclusiones	29
	GLOSARIO	30
	RESUMEN ESQUEMÁTICO	31
	PREGUNTAS DE TEST	34
	SOLUCIONES A LAS PREGUNTAS DE TEST	36

1 Introducción a las aplicaciones móviles

Las aplicaciones móviles (apps) son aplicaciones informáticas diseñadas para ser ejecutadas en teléfonos inteligentes, tabletas y otro tipo de dispositivos móviles. El origen de las primeras aplicaciones móviles se remonta a los años 90, cuando aparecen aplicaciones muy básicas como la agenda, los calendarios, algunos videojuegos (como el Tetris o el famoso Snake), o las aplicaciones de correo electrónico. Posteriormente, ya en los años 2000, las aplicaciones comenzaron a evolucionar más rápidamente, debido sobre todo a la irrupción de la tecnología WAP (Wireless Application Protocol), que permitió el acceso a Internet desde los dispositivos móviles.

Sin embargo, el verdadero auge de las aplicaciones móviles comienza en el año **2007**, cuando la compañía Apple lanza al mercado su **primer iPhone**. Este inicialmente no tenía soporte para instalar aplicaciones adicionales a las que contenía por defecto, no obstante, al año siguiente, Apple inaugura la App Store con unas 500 aplicaciones disponibles para ser descargadas. Además, Apple facilitó a los desarrolladores su SDK (Software Development Kit), con el objetivo de que cualquiera pudiese crear una aplicación móvil y subirla a la App Store, para que posteriormente fuera descargada desde los dispositivos iPhone. La idea tuvo una gran acogida entre el público, y en el año 2009 la App Store contenía ya más de 50.000 aplicaciones de terceros, y se habían realizado más de 25.000 millones de descargas.

Por otra parte, durante el año **2008** también se produce el despegue de **Android**, ya que se publica el primer Android SDK, y posteriormente se lleva a cabo la apertura del Android Market (actualmente Google Play), que contenía inicialmente 50 aplicaciones móviles listas para ser descargadas a los dispositivos con sistema operativo Android.

Desde entonces, la venta de teléfonos inteligentes y el uso de aplicaciones móviles no ha parado de crecer. Actualmente, Google Play cuenta con unos 2,8 millones de aplicaciones, y App Store con alrededor de 2,2 millones. Las posibilidades que ofrecen las aplicaciones móviles hoy en día son innumerables, pues existen aplicaciones dirigidas a todo tipo de público y todo tipo de sectores (educación, deporte, política, medicina etc.). La Administración Pública no ha sido ajena a este cambio, y actualmente disponen también de un amplio conjunto de aplicaciones móviles como CI@ve PIN de la Agencia Tributaria, AlertCops de la Policía Nacional y la Guardia Civil para emitir avisos de seguridad ciudadana, o la aplicación de la Dirección General de Tráfico para conocer las incidencias en las carreteras españolas.

2 Características y tecnologías

Dado el gran auge que han experimentado las aplicaciones móviles en la última década, no es de extrañar la continua evolución tecnológica que se produce en este campo. De hecho, actualmente existen tres aproximaciones distintas para afrontar el desarrollo de aplicaciones móviles, cada una de las cuales utiliza diferentes tecnologías y herramientas. Estas tres aproximaciones son las siguientes:

- Desarrollo de aplicaciones web móviles.
- Desarrollo de aplicaciones nativas.
- Desarrollo de aplicaciones híbridas.

A lo largo de este capítulo se describen estas tres alternativas, sus características, sus ventajas e inconvenientes, así como las principales tecnologías utilizadas por cada una de ellas. A la hora de seleccionar la aproximación a utilizar, es importante tener en cuenta las necesidades y restricciones del proyecto a abordar, ya que esto hará que unas opciones sean más apropiadas que otras para ese proyecto en concreto.

2.1 Aplicaciones web móviles

Este tipo de aplicaciones móviles se desarrollan con tecnologías web como **HTML, CSS y Javascript**, de modo similar a como se han venido desarrollando hasta ahora las aplicaciones web. La idea consiste en utilizar los conocimientos que ya se tienen sobre desarrollo web, y utilizarlos para crear aplicaciones que puedan mostrar los contenidos no solo en los dispositivos tradicionales, sino también en los dispositivos móviles. Para ello, las aplicaciones **se cargan a través del navegador web del dispositivo**, lo cual significa que **no es necesaria la compilación ni la instalación** de la aplicación en el dispositivo final.

En 2007, cuando salieron al mercado los primeros iPhone, la única forma de crear aplicaciones móviles de terceros era ésta, puesto que todavía no existía la App Store, y por lo tanto las únicas aplicaciones disponibles eran las preinstaladas en los dispositivos o las accesibles a través de navegador web. Sin embargo, estos teléfonos tenían todavía grandes desventajas, ya que sus navegadores web eran poco potentes, y las velocidades de descarga bastante reducidas. Además, las aplicaciones web se diseñaban para ser visualizadas en pantallas de mayor tamaño, y no en pequeñas pantallas como las de los dispositivos móviles. Por estos motivos, las aplicaciones web móviles del momento no proporcionaban una buena experiencia de usuario.

Para tratar de solucionar este problema se avanzó en dos sentidos. En primer lugar, iPhone comenzó a mejorar la visualización y la experiencia de usuario en los navegadores web de sus dispositivos, lo que tuvo una buena acogida por parte de los usuarios, y animó a su vez a otros fabricantes a optimizar sus navegadores. Por otra parte, muchas veces se desarrollaban varias aplicaciones con el mismo contenido, pero cada una de ellas pensada para un tamaño de pantalla.

Sin embargo, la rápida proliferación de dispositivos móviles de distintos fabricantes, y con diferentes tamaños de pantalla, produjo la necesidad de buscar otra alternativa, ya que desarrollar la misma aplicación varias veces para ser visualizada en diversos teléfonos era demasiado costoso en tiempo y dinero, e implicaba demasiadas duplicidades. Así fue como aparecieron las **media queries**. Éstas permiten determinar cuál es el tamaño del dispositivo que se está utilizando, y en función de ello ajustar la visualización de la aplicación a través de estilos CSS (Cascading Style Sheets): mostrar más o menos columnas, utilizar letra de mayor o menor tamaño etc. Sin embargo, las media queries no funcionaron bien hasta que los navegadores comenzaron a soportar la versión 3 de CSS (CSS3) en el año 2009.

Al año siguiente, en 2010, basándose en las capacidades de adaptación a los dispositivos proporcionadas por las media queries, así como en el concepto de flexibilidad (en el sentido de ajuste de los tamaños de los elementos de interfaz de usuario en porcentajes en lugar de en medidas fijas, posicionamiento relativo en lugar de absoluto etc.) surge el concepto de **Responsive Web Design** (RWD) o diseño web adaptativo.

El Responsive Web Design es un conjunto de técnicas que permiten que las aplicaciones web puedan ser visualizadas correctamente en diversos tipos de dispositivos y para diversos tamaños de pantalla (desde los teléfonos móviles de menor tamaño hasta los grandes monitores de escritorio), proporcionando un rendimiento adecuado y una buena experiencia de usuario.

Una aplicación web que ha sido creada utilizando RWD es capaz de adaptar su visualización al entorno en el que se ejecuta, y para ello se apoya en tres elementos fundamentales:

- Uso de Media Queries.
- Diseño fluido: consiste en definir el tamaño de los elementos de interfaz (campos de texto, listas de selección etc.) en base a unidades relativas (como porcentajes), en lugar de utilizar tamaños fijos. De esta forma, es posible adaptar los elementos a la pantalla de forma proporcional a su tamaño.
- Elementos flexibles, como por ejemplo las fuentes, las imágenes o los elementos multimedia, que también adaptarán su tamaño de forma proporcional.

Ya en el año 2014, se produce otro avance importante en el diseño de aplicaciones web móviles, provocado por la publicación de la versión 5 de HTML (**HTML5**). Esta versión introduce importantes novedades con respecto a las anteriores, y muchas de ellas son de gran ayuda a la hora de conseguir aplicaciones móviles que proporcionen una buena experiencia de usuario. En concreto, destacan las siguientes:

- HTML5 es adaptable, es decir, permite realizar diseños que se adaptan a diferentes dispositivos (web, tabletas, móviles...) y ofrece una mayor compatibilidad con los navegadores de distintos dispositivos.
- Simplifica el código provocando páginas más ligeras y, por tanto, más rápidas, algo que es fundamental a la hora de visualizar adecuadamente las aplicaciones en los dispositivos móviles.
- Incorpora nuevas capacidades JavaScript, como la posibilidad de ejecutar múltiples JavaScript en paralelo en una misma página y ejecutar páginas sin estar conectado.
- Mejora el soporte a contenido multimedia, posibilitando la inserción directa de vídeos y audio.
- Reduce la necesidad del uso de plugins, como por ejemplo Flash.
- Introduce nuevas APIs, entre las que destacan las siguientes:
 - HTML Geolocation: usada para obtener la información geográfica del usuario. Muchas aplicaciones móviles de hoy en día necesitan esta característica (por ejemplo, Google Maps, Waze, El Tiempo de AEMET etc.)
 - HTML Drag and Drop: permite arrastrar y soltar objetos en otra localización de la pantalla.
 - HTML Application Cache: permite cachear la aplicación, de modo que posteriormente se puede acceder a ella en modo offline, reduciéndose así el tiempo de carga y mejorando la velocidad.
 - HTML Web Workers: un web worker es un JavaScript que se ejecuta en segundo plano (background), de forma independiente a otros scripts. Esto evita bloqueos de espera y permite continuar navegando por la página mientras el web worker se ejecuta.
 - HTML SSE (Server-Sent Events): permite actualizar una página web automáticamente sin necesidad de tener que preguntar al servidor si hay algún cambio disponible.

Actualmente, el Responsive Web Design y el desarrollo móvil siguen creciendo en importancia, ya que cada vez más usuarios acceden a los contenidos a través de dispositivos móviles (se estima que en la actualidad más de la mitad del tráfico de internet se produce a través de este tipo de dispositivos). De hecho, en 2015 Google lanzó **Mobiledgeddon**, que incluye un algoritmo de ranking de páginas web pensado para ser utilizado desde los teléfonos inteligentes y tabletas, ya que proporciona mayor puntuación a las páginas que están adaptadas para ser visualizadas correctamente en este tipo de dispositivos, por lo que aparecen en los primeros puestos tras realizar una búsqueda.

Por otra parte, cabe destacar que, a pesar de que a lo largo de esta última década han ido surgiendo nuevos paradigmas de desarrollo móvil, esta aproximación mediante el desarrollo web móvil sigue siendo válida, y de hecho, resulta especialmente útil en ciertos casos, como cuando se necesita disponer de una aplicación móvil en un período muy breve de tiempo y con un presupuesto reducido. Sin embargo, antes de elegir esta opción, conviene revisar las características concretas del proyecto, ya que, en función de éstas, puede ser necesario llevar a cabo un desarrollo utilizando alguno de los otros modelos que se expondrán en los siguientes apartados.

A modo de resumen, se presentan ahora las principales **ventajas** del desarrollo web móvil:

- Menor coste.
- Facilidad de desarrollo, al utilizar tecnologías web ampliamente conocidas.
- Un único desarrollo permite abarcar todas las plataformas: Android, iOS, Windows, televisiones, videoconsolas etc.
- No son necesarios conocimientos específicos de cada una de las plataformas.
- Las aplicaciones no requieren instalación, puesto que se accede a ellas a través del navegador web del dispositivo.
- Las actualizaciones de la aplicación son transparentes para los usuarios.

Sin embargo, esta aproximación también presenta ciertos inconvenientes, entre los que destacan:

- La experiencia de usuario y la usabilidad son peores que para las aplicaciones nativas.
- Peor rendimiento.
- No permite el acceso a funciones del sistema operativo del dispositivo como la cámara, la geolocalización, el acelerómetro, el acceso a ficheros etc.
- A pesar de que con el uso de cachés es posible realizar algunas funciones en modo offline, en general se requiere una conexión de datos en el dispositivo para poder ejecutar la aplicación.

Para finalizar este apartado, se muestran un conjunto de **escenarios típicos** donde este tipo de desarrollo parece el más recomendable:

- Creación de pilotos, con el objetivo de determinar si una determinada aplicación podría tener éxito en el mercado, ya que dicho piloto se podría crear rápidamente y con un bajo coste.
- Se dispone de bajo presupuesto y poco tiempo.
- Es necesario utilizar la aplicación en todo tipo de dispositivos: teléfonos inteligentes, tabletas, videoconsolas, televisores, etc.

2.2 Aplicaciones nativas

Las aplicaciones nativas están **desarrolladas y optimizadas para ser utilizadas en un sistema operativo** en particular (Android, iOS etc.), adaptándose perfectamente a las funcionalidades y características del dispositivo, de modo que se obtiene una **excelente experiencia de usuario**.

Al contrario que las aplicaciones web móviles, a este tipo de aplicaciones NO se accede a través del navegador, sino que han de ser **descargadas, instaladas y almacenadas** en el dispositivo. Para ello, el usuario accede a la tienda de aplicaciones de la correspondiente plataforma (Google Play para Android, Apple Store para iOS etc.), busca la aplicación de su interés, e inicia la descarga. Tras la inicialización, la aplicación nativa se conecta directamente con el sistema operativo móvil, sin ningún intermediario ni contenedor, y puede acceder a todas las APIs (Application Programming Interface) que el proveedor del sistema operativo ponga a disposición.

A la hora de crear estas aplicaciones, es necesario utilizar el **kit de desarrollo específico para la plataforma** deseada, conocido como **SDK** (Software Development Kit). Un SDK es, por lo tanto, un conjunto de herramientas que permiten el desarrollo para un sistema operativo en concreto, como Android o iOS. Como ya se ha comentado anteriormente, los primeros SDKs aparecieron en el año 2008, por lo que previamente los desarrollos estaban restringidos a las aplicaciones web móviles. En

cambio, actualmente existen un gran número de aplicaciones creadas mediante desarrollo nativo, como por ejemplo Whatsapp o Facebook.

A continuación se resumen las principales **ventajas** que proporciona el desarrollo nativo, ventajas que son independientes del sistema operativo concreto para el que se programe:

- Buena usabilidad.
- Riqueza de la interfaz de usuario.
- Buen rendimiento.
- Acceso a las funciones del sistema operativo del dispositivo: cámara, geolocalización, acelerómetro, acceso a ficheros etc.
- Pueden trabajar en modo offline, y están disponibles incluso con el modo avión activado.

Sin embargo, esta aproximación también presenta algunos **inconvenientes**, entre los que se encuentran los siguientes:

- Es la alternativa que implica un mayor coste.
- Se requiere un desarrollo para cada una de las plataformas en las que vaya a utilizarse la aplicación. En general, para abarcar la mayoría de dispositivos del mercado es necesario desarrollar al menos para Android e iOS.
- Se requiere disponer de personal con conocimientos de todas las plataformas en las que se vaya a desarrollar.
- Se requiere completar un proceso de revisión y aprobación para que las aplicaciones están disponibles en las respectivas tiendas: Google Play, App Store etc.
- Las aplicaciones requieren instalación.
- Los usuarios han de instalar también las actualizaciones, y éstas requieren de nuevo un proceso de revisión y actualización.

Debido a estas ventajas e inconvenientes característicos del desarrollo nativo, existen un conjunto de **escenarios típicos** en los que este enfoque de desarrollo puede resultar más apropiado. En concreto, esta aproximación puede ser de gran utilidad cuando:

- Solo se necesita desarrollar para una plataforma, por ejemplo, porque es una aplicación interna que solo va a ser utilizada por trabajadores de la empresa, cuyos dispositivos son todos del mismo fabricante.
- El núcleo de la funcionalidad requiere acceso a las funciones del sistema operativo del dispositivo.
- Se requiere una interfaz de usuario sofisticada. Esto puede ser necesario por ejemplo para el desarrollo de algunos juegos.
- Se dispone de trabajadores con conocimientos en desarrollo nativo.

Una vez comentadas las características, ventajas e inconvenientes del desarrollo nativo en general, se procede a detallar un poco más el proceso de desarrollo para los tres sistemas operativos móviles más importantes a día de hoy: Android, iOS y Windows. Además, se hará referencia también a la posibilidad de desarrollo nativo multiplataforma, para lo cual se puede utilizar Xamarin, un API de Microsoft que permite desarrollar aplicaciones móviles para diversos sistemas operativos utilizando .NET.

2.2.1 Aplicaciones Android

Android es un **sistema operativo de código abierto basado en Linux**, diseñado para ser utilizado en una amplia variedad de dispositivos. Inicialmente fue desarrollado por Android Inc., empresa que **Google** respaldó económicamente y más tarde, en 2005, compró.

El desarrollo de aplicaciones nativas para Android se realiza normalmente utilizando lenguaje **Java** y el **Android SDK** provisto por Google. Éste contiene un conjunto de herramientas que permiten compilar el código fuente, añadir archivos de recursos (imágenes, iconos...) y de datos, y crear un paquete con extensión **apk** que será utilizado para instalar la aplicación.

En los siguientes apartados se profundizará en algunos de los conceptos más importantes sobre el desarrollo nativo para Android. Por ello, se expondrá cuál es la arquitectura de la plataforma Android, cuáles son los componentes que forman parte de una aplicación Android, y cuáles son los principales entornos a utilizar durante el desarrollo.

2.2.1.1 Arquitectura de la plataforma Android

Android es una pila de software de código abierto basado en Linux, formada por un conjunto de capas, cada una de las cuales utiliza los servicios ofrecidos por las capas inferiores, al mismo tiempo que proporciona servicios a las capas superiores. La arquitectura de la plataforma Android se puede ver en la siguiente figura:

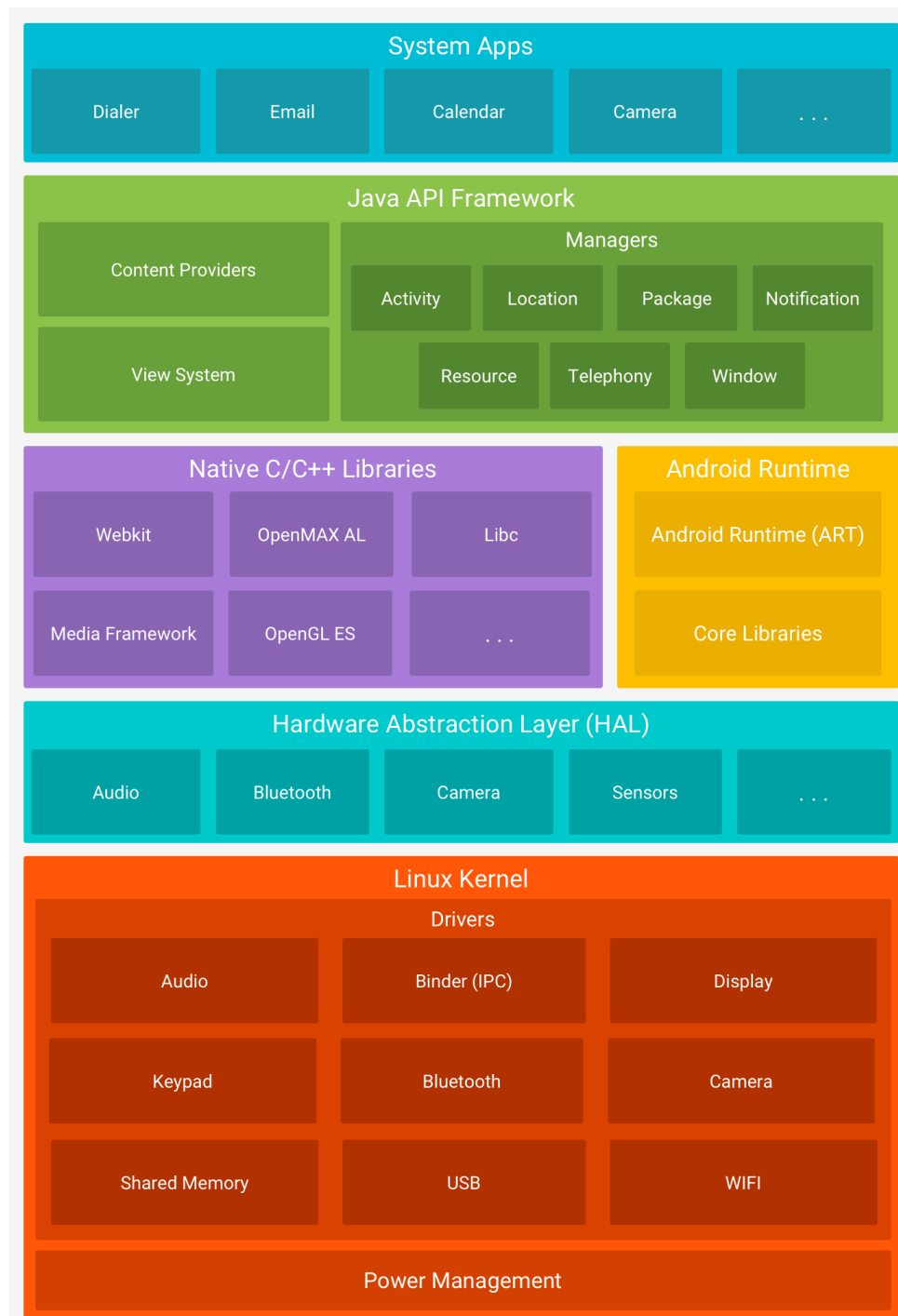


Figura 1. Arquitectura de la plataforma Android

A continuación se describen, de abajo hacia arriba, los componentes que integran esta arquitectura:

- **Núcleo/Kernel de Linux:** la base de la plataforma Android es el núcleo (kernel) de Linux, lo que permite que Android reutilice funciones clave (gestión de memoria a bajo nivel, elementos de seguridad etc.), al mismo tiempo que permite a los fabricantes de dispositivos desarrollar controladores de hardware para un kernel conocido.
- **Capa de abstracción de hardware (HAL: Hardware Abstraction Layer):** consiste en un conjunto de módulos, cada uno de los cuales implementa una interfaz para un tipo específico de componente hardware (cámara, Bluetooth etc.). Cuando se necesita acceder al hardware del dispositivo, Android carga el módulo apropiado para el componente hardware en cuestión.
- **Android Runtime:**
 - **ART:** tiempo de ejecución de Android, utilizado en dispositivos con Android 5.0 (API 21) o posterior. Entre sus características, se encuentra la posibilidad de realizar compilación anticipada (AOT: Ahead of Time) o en tiempo de ejecución (JIT: Just In Time), la mejora en la recolección de elementos no utilizados, la mejora de las características de depuración etc. Las versiones de Android anteriores a la 5.0 utilizaban Dalvik en lugar de ART como tiempo de ejecución.
 - **Core Libraries:** conjunto de librerías de tiempo de ejecución centrales, que proporcionan la mayor parte de la funcionalidad del lenguaje de programación Java.
- **Bibliotecas C/C++ nativas:** muchos componentes y servicios centrales del sistema Android, como el ART y la HAL, se basan en código nativo que requiere bibliotecas nativas escritas en C y C++.
- **Framework de la API Java:** todo el conjunto de funciones del sistema operativo Android está disponible mediante APIs escritas en el lenguaje Java. Por otra parte, muchas de las bibliotecas C/C++ nativas descritas anteriormente también se exponen mediante este framework.
Un ejemplo de uso de estas APIs, es el acceso a un administrador de notificaciones que permite a las aplicaciones mostrar alertas personalizadas en la barra de estado.
- **Aplicaciones (apps) del sistema:** se trata de un conjunto de aplicaciones como correo electrónico, calendarios, navegación en internet, contactos etc. que proporcionan capacidades clave a las que los desarrolladores pueden acceder desde sus propias aplicaciones.
Es decir, una app nativa puede hacer uso de la aplicación de sistema de correo electrónico, evitando de esta forma desarrollar la funcionalidad de correo dentro de la propia aplicación.

2.2.1.2 Componentes

Las aplicaciones Android están formadas por uno o más de los siguientes **componentes**:

- **Actividad:** representa una pantalla con interfaz de usuario. Diversas actividades pueden trabajar conjuntamente para proporcionar una experiencia de usuario consistente, pero cada actividad siempre es independiente de las demás.
Por ejemplo, una aplicación de correo electrónico tiene una actividad que muestra una lista de los correos electrónicos nuevos, otra para redactar el correo y una tercera para leer correos. La independencia de las actividades permitirá a una aplicación de cámara iniciar la actividad de redacción de correos con el objetivo de compartir la imagen capturada.
- **Servicio:** se ejecuta en segundo plano para realizar operaciones prolongadas o tareas para procesos remotos, y NO proporciona una interfaz de usuario. Otro componente, como una actividad, puede iniciar el servicio y permitir que se ejecute o enlazarse a él para interactuar. Por ejemplo, un servicio podría reproducir música en segundo plano mientras el usuario se encuentra en otra aplicación.

- **Proveedor de contenido:** administra un conjunto compartido de datos de la aplicación. Permite almacenar los datos en el sistema de archivos, en una base de datos SQLite, en la Web o en cualquier otra ubicación de almacenamiento persistente a la que se tenga acceso. Además, a través del proveedor de contenido, otras aplicaciones pueden consultar o incluso modificar los datos.

Como ejemplo, el sistema Android proporciona un proveedor de contenido que administra la información de contacto del usuario. De esta manera, cualquier aplicación con los permisos necesarios puede consultar parte del proveedor de contenido para la lectura y escritura de información sobre una persona específica.

- **Receptor de mensajes:** estos componentes responden a eventos en todo el sistema, como pueden ser los eventos de batería baja, pérdida de conexión WIFI, reinicio del sistema etc. Si bien no tienen interfaz de usuario, pueden crear una notificación en la barra de estado para alertar al usuario.

En cuanto a la **forma de activar estos componentes**, los proveedores de contenidos se activan mediante solicitudes de un solucionador de contenido. El solucionador actúa como capa de abstracción entre el componente que solicita la información y el proveedor del contenido, de modo que no existan problemas de seguridad.

En cambio, los otros tres componentes se activan mediante un mensaje asíncrono llamado Intent. Para actividades y servicios, el Intent define la acción a realizar (por ejemplo, "ver" o "enviar" algo) y puede especificar el URI de los datos en los que debe actuar. Por ejemplo, un Intent podría transmitir una solicitud para que una actividad muestre una imagen o abra una página web.

Para los receptores de mensajes, el Intent simplemente define el anuncio que se está transmitiendo (por ejemplo, un mensaje para indicar que la batería del dispositivo tiene poca carga incluye solo una cadena de caracteres de acción conocida que indica "batería baja").

Por otra parte, para que Android pueda iniciar un componente de la aplicación, el sistema debe reconocer la existencia de ese componente leyendo el **archivo de manifiesto** (archivo **AndroidManifest.xml**), que debe encontrarse en la raíz del directorio de proyectos de la aplicación.

Además de declarar los componentes de la aplicación, el manifiesto permite también:

- Identificar los permisos de usuario que requiere la aplicación, como por ejemplo los permisos de acceso a Internet o de lectura de los contactos.

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

- Declarar el nivel de API mínimo requerido por la aplicación, ya que ésta puede no ser compatible con versiones antiguas de Android.

```
<uses-sdk android:minSdkVersion="4" android:targetSdkVersion="15" />
```

- Declarar características de hardware y software que la aplicación usa o exige, como una cámara, servicios de Bluetooth o una pantalla multitáctil.

```
<uses-feature android:name="android.hardware.bluetooth" />
<uses-feature android:name="android.hardware.camera" />
```

- Etc.

2.2.1.3 Entornos de desarrollo

Hasta el año 2014, Google recomendaba desarrollar las aplicaciones nativas Android con el entorno de desarrollo Eclipse, al que se añadía un plugin para Android (plugin ADT, que posteriormente se sustituyó por el plugin AndMore).

Sin embargo, a finales de 2014 Google lanzó la primera versión estable de **Android Studio**, que desde ese momento se ha convertido en el entorno de desarrollo oficial para la plataforma Android. Android Studio es un IDE basado en IntelliJ, que proporciona, entre otras, las siguientes funciones:

- Potente editor de código.
- Emuladores, utilizados para probar la aplicación móvil simulando su ejecución en distintos tipos de dispositivos y con diferentes sistemas operativos.
- Herramientas y frameworks de prueba.
- Herramientas para detectar problemas de rendimiento, usabilidad, compatibilidad de versiones etc.
- Soporte incorporado para Google Cloud Platform, lo que facilita la integración de Google Cloud Messaging y App Engine.
- Instant Run para aplicar cambios mientras la aplicación se ejecuta, sin necesidad de volver a compilar.
- Etc.

Actualmente sigue siendo posible desarrollar con Eclipse for Android Developers, o incluso con otros entornos de desarrollo Java a los que se añaden plugins, como por ejemplo NetBeans. Sin embargo, la recomendación oficial consiste en utilizar Android Studio, puesto que este IDE está completamente adaptado al desarrollo para todo tipo de dispositivos Android.

2.2.2 Aplicaciones iOS

El sistema operativo iOS (inicialmente denominado iPhone OS) es un sistema operativo desarrollado por **Apple** para ser **utilizado de forma exclusiva en sus dispositivos (iPad, iPhone y iPod)**. iOS se encarga de gestionar el hardware del dispositivo y proporciona las tecnologías necesarias para implementar aplicaciones nativas. Estas aplicaciones se desarrollan haciendo uso del **iOS SDK**, que contiene las herramientas e interfaces requeridas para desarrollar, instalar, ejecutar y probar las aplicaciones. Para la codificación de aplicaciones iOS normalmente se utiliza el lenguaje **Swift** (aunque todavía existen aplicaciones desarrolladas con lenguaje **Objective-C**). Además, muchas veces las aplicaciones hacen uso de diversos **frameworks**, que no son más que un directorio que contiene una librería compartida dinámica junto con los recursos necesarios para soportarla. Finalmente, cuando las aplicaciones se empaquetan para ser distribuidas en la App Store, se obtienen paquetes con extensión **ipa**.

En los siguientes apartados se profundizará en algunos de los conceptos más importantes sobre el desarrollo nativo para iOS. Por ello, se expondrá cuál es la arquitectura de la plataforma iOS, cuáles son los objetos más importantes que se utilizan durante el desarrollo, y cuáles son los principales entornos a utilizar durante el desarrollo.

2.2.2.1 Arquitectura de la plataforma iOS

La arquitectura de la plataforma iOS está formada por el conjunto de capas que se pueden ver en la figura que se muestra más abajo. Las capas más bajas contienen servicios y tecnologías básicas, mientras que las capas de alto nivel hacen uso de las capas inferiores y proporcionan servicios y tecnologías más avanzados. A la hora de desarrollar aplicaciones, se recomienda utilizar las capas de alto nivel siempre que sea posible, puesto que generalmente reducen la cantidad de código necesario y encapsulan funcionalidades complejas.

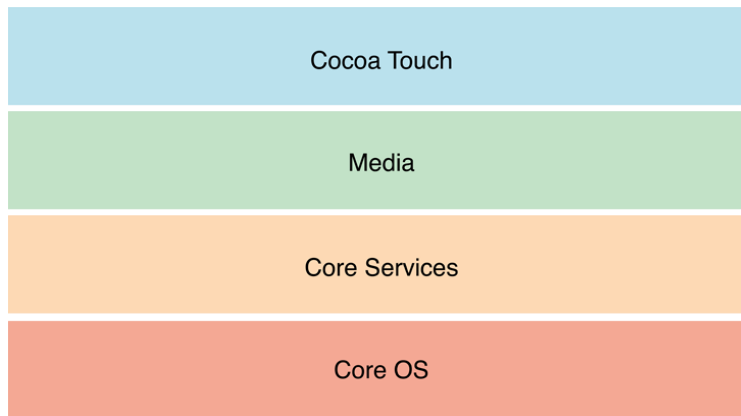


Figura 2. Arquitectura de la plataforma iOS

A continuación se describen, de abajo hacia arriba, los componentes que integran esta arquitectura:

- **Núcleo del sistema operativo (Core OS Layer):** contiene funcionalidades de bajo nivel que usan la mayor parte de las capas superiores. Aunque en general no se hace un uso directo de esta capa durante el desarrollo, puede que se requiera interactuar con ella si por ejemplo la aplicación ha de gestionar la comunicación con un accesorio hardware externo.
- **Capa de servicios del núcleo (Core Services Layer):** contiene servicios básicos para las aplicaciones. Entre ellos destacan los frameworks Foundation y Core Foundation, que definen los tipos de datos básicos que todas las aplicaciones móviles utilizan. Además, esta capa también contiene funcionalidades relativas a redes sociales, localización, iCloud y gestión de red.
- **Capa multimedia (Media Layer):** contiene tecnologías de audio, vídeo y gráficos.
- **Capa Cocoa Touch (Cocoa Touch Layer):** proporciona la infraestructura básica de la aplicación y el soporte para tecnologías clave como la gestión multitarea, la entrada táctil o las notificaciones push. También permite definir la apariencia de las apps.

2.2.2.2 Objetos más importantes en una aplicación iOS

Los objetos más importantes que se utilizan en una aplicación iOS son los siguientes:

- **UIApplication:** gestiona comportamientos de alto nivel de la aplicación, y reporta eventos (como por ejemplo las notificaciones push) a su objeto delegado.
- **App Delegate:** gestiona (en colaboración con el objeto UIApplication) la inicialización de la aplicación, las transiciones de estado, y muchos otros eventos de alto nivel de la aplicación.
- **Objetos de modelo de datos (Data Model Objects):** almacenan los contenidos, y son específicos de la aplicación (por ejemplo, una aplicación de banca almacenará una base de datos con transacciones financieras).
- **Objetos Vista Controlador (UIViewController Objects):** gestionan la presentación de la aplicación en pantalla.
- **Ventanas (UIWindow Objects):** coordina la presentación de una o más vistas en la pantalla. La mayoría de aplicaciones solo contienen una ventana, que presenta el contenido en la pantalla principal; pero algunas aplicaciones pueden requerir otra ventana para visualizar los contenidos en un dispositivo externo.

- **Vistas (UIView Objects):** objeto que dibuja el contenido en un área rectangular y responde a los eventos que se producen en esa área.
- **Controles (UIControl Objects):** son un tipo especializado de vistas, que se encargan de implementar objetos de interfaz comunes como botones o campos de texto.

Por otra parte, las aplicaciones iOS utilizan el fichero **Info.plist**, un archivo que contiene información de configuración almacenada en formato XML, a la que el sistema puede acceder en tiempo de ejecución. El nodo raíz es un diccionario, que contiene una lista de claves y valores que describen diferentes aspectos de la aplicación. Por ejemplo, se puede utilizar este fichero para determinar el icono a utilizar cuando se lanza la aplicación o los tipos de documentos que la app soporta.

2.2.2.3 Entornos de desarrollo

El desarrollo de aplicaciones iOS (así como el desarrollo para Mac, Apple Watch y Apple TV) se lleva a cabo utilizando el entorno de desarrollo **XCode**. XCode es un IDE que proporciona todas las herramientas necesarias para gestionar el flujo completo de desarrollo de aplicaciones iOS, incluyendo la codificación, el diseño de interfaces de usuario, la optimización de código, la realización de pruebas y el envío de la aplicación a la App Store. Los lenguajes de programación que habitualmente se utilizan son **Objective-C**, y más recientemente **Swift**, si bien es cierto que XCode puede compilar también otros lenguajes como C, C++, Java o AppleScript.

Las principales **funcionalidades** ofrecidas por XCode son:

- Potente editor de código.
- Catálogo de recursos, que permite gestionar fácilmente las imágenes utilizadas por la aplicación.
- Integración con herramientas de control de versiones.
- Simulador que facilita el testeo de las aplicaciones.
- Herramientas para diseñar la interfaz de usuario.
- Herramientas de pruebas y análisis de código.
- Integración continua.
- Preparación y envío de aplicaciones a la App Store.
- Etc.

Además, XCode incluye la herramienta **Interface Builder**, que permite crear interfaces de usuario de forma fácil y rápida, arrastrando y soltando elementos de interfaz (como botones, campos de texto etc.) dentro del diseñador. Y también cuenta con un asistente que permite conectar los controles de interfaz de usuario con el código que implementa su comportamiento.

Asimismo, con XCode es posible gestionar **Storyboards**, los cuales definen las relaciones entre las vistas de la aplicación. Las apps iOS se componen de múltiples vistas a través de las cuales navegan los usuarios, y los storyboards permiten gestionar estas navegaciones.

A la hora de probar las interfaces de usuario diseñadas, se puede hacer uso del modo *Vista previa*, que permite visualizar rápidamente dichas interfaces en una gran variedad de situaciones (orientación horizontal o vertical, diferentes versiones del sistema operativo iOS, diversos tamaños de pantalla etc.).

Por otra parte, XCode se integra con una potente herramienta que permite realizar pruebas y analizar el rendimiento de la aplicación, conocida como **Instruments**. Esta herramienta permite examinar el comportamiento de la app, detectar problemas de memoria, optimizar el rendimiento, localizar código fuente con posibles errores, e incluso examinar características específicas del dispositivo como la conexión WIFI o el Bluetooth.

2.2.3 Aplicaciones Windows

El sistema operativo Windows 8 introdujo Windows Runtime (WinRT), una arquitectura de aplicaciones común, independiente de la plataforma, y que permitía el desarrollo utilizando diferentes lenguajes de programación como C++, JavaScript-TypeScript, C# o Visual Basic.NET. Cuando posteriormente se publicó Windows Phone 8.1, se creó una versión de WinRT para ser utilizada en los dispositivos móviles, conocida como Windows Phone Runtime. De este modo, los desarrolladores comenzaron a crear aplicaciones universales para Windows 8 destinadas tanto a sistema operativo Windows como a Windows Phone, sobre la base de un código común compartido.

Partiendo de esta idea de reutilización de código entre plataformas, **Microsoft** publicó en el año 2015 el sistema operativo Windows 10, con el cual se unifica el desarrollo de aplicaciones para diferentes familias de dispositivos (aplicaciones de escritorio, dispositivos móviles, dispositivos IoT etc.) mediante el uso de la **Plataforma Universal de Windows** (UWP: Universal Windows Platform). Con UWP, se incorpora el modelo de Windows Runtime al núcleo de Windows 10, de modo que se proporciona una plataforma de aplicaciones común, disponible en todos los dispositivos que se ejecutan en Windows 10. Además, esto también supuso la desaparición de Windows Phone, puesto que ahora los dispositivos móviles utilizan una edición del sistema operativo Windows 10 conocida como **Windows 10 Mobile**.

En los siguientes apartados se profundizará en algunos de los conceptos más importantes sobre el desarrollo nativo para dispositivos móviles Windows. Para ello, se profundizará en las características de la Plataforma Universal de Windows, y se explicará cuál es el entorno de desarrollo a utilizar a la hora de construir este tipo de aplicaciones.

2.2.3.1 La Plataforma Universal de Windows (UWP: Universal Windows Platform)

Como ya se ha comentado anteriormente, la Plataforma Universal de Windows permite desarrollar aplicaciones que funcionan en diferentes familias de dispositivos. Una **familia de dispositivos** identifica las APIs, las características del sistema y los comportamientos de todos los dispositivos que forman parte de esa familia. En la siguiente figura se muestran las familias que existen:

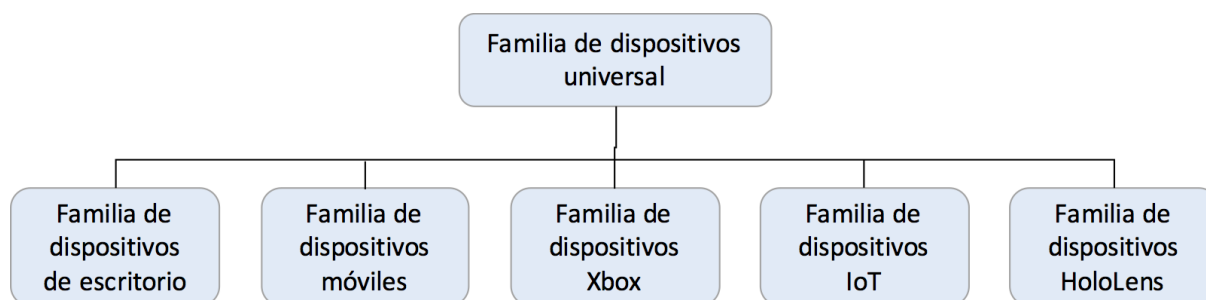


Figura 3. Familias de dispositivos en la Plataforma Universal de Windows

Los dispositivos móviles (teléfonos, tabletas etc.) ejecutan el sistema operativo móvil, que está basado en la familia de dispositivos móviles que se puede ver en la figura anterior; al igual que los ordenadores ejecutan el sistema operativo de escritorio basado en la familia de dispositivos de escritorio.

Además, existe una familia de dispositivos especial que se encuentra en la base de la jerarquía: la familia de dispositivos universal. Esta familia, al contrario que todas las demás, no es la base directa de un sistema operativo, sino que proporciona un conjunto de APIs que heredan el resto de familias (las familias secundarias), de modo que se asegura que las APIs de la familia de dispositivos universal están presentes en todos los sistemas operativos y en todos los dispositivos. Además, cada familia de dispositivos secundarios agrega sus propias APIs a las que hereda.

Las principales **ventajas** que presenta la Plataforma Universal de Windows frente a los anteriores modelos de desarrollo son las siguientes:

- Existe un API común para todos los dispositivos, por lo que si **las aplicaciones** desarrolladas solo utilizan las APIs principales, **se podrán ejecutar en cualquier tipo de dispositivo Windows**: teléfonos, tabletas, equipos de escritorio, consolas Xbox etc.
- Es posible añadir **características adicionales** a las aplicaciones **para cada tipo de dispositivos, utilizando** para ello **SDKs de extensión**, que agregan APIs especializadas con las que se mejora la experiencia de usuario.
- Todas las aplicaciones, independientemente del dispositivo al que se dirijan, se distribuyen en **paquetes AppX** y se pueden descargar de la **Tienda Windows**. De este modo, es posible administrar todas las aplicaciones en una sola tienda, e indicar para cada una de ellas, en qué familia o familias de dispositivos va a estar disponible.
- Es posible desarrollar utilizando **diferentes lenguajes de programación**, como C#, Visual Basic, C++ o Javascript. Para la parte de interfaz de usuario también existen varias opciones como XAML, DirectX si se utiliza C++, o HTML si se utiliza Javascript.
- Los elementos de interfaz de usuario pueden ser adaptados al número de píxeles de pantalla disponibles en el dispositivo.

Por otra parte, a la hora de **decidir a qué familia o familias de dispositivos se va a dirigir la aplicación** es conveniente tener en cuenta las siguientes recomendaciones:

- Si lo que se busca es conseguir que la aplicación se ejecute **en todo tipo de dispositivos**, se utilizará la **familia de dispositivos universal**.
- Si la aplicación **solo** va a estar **disponible para cierto tipo de dispositivos**, entonces se utilizará la **familia secundaria correspondiente a esos dispositivos**. Por ejemplo, para desarrollar aplicaciones móviles se utiliza la familia de dispositivos móviles, y para desarrollar para Xbox la familia de dispositivos Xbox. Esto tiene la ventaja de que el número de APIs disponibles es mayor, al incluirse no solo las APIs de la familia universal, sino también las heredadas. De este modo, la interfaz de usuario es más rica y se mejora la experiencia de usuario.
- Si se pretende que la aplicación funcione **en varios tipos de dispositivos, pero no en todos**, entonces se destinará al **conjunto de familias secundarias correspondientes a esos dispositivos**. Por ejemplo, si la aplicación ha de estar disponible tanto en dispositivos móviles como de escritorio, entonces se utilizarán ambas familias.

2.2.3.2 Entornos de desarrollo

Para desarrollar aplicaciones utilizando UWP se utiliza el IDE **Visual Studio**, en el que hay que asegurarse que están instaladas las herramientas para el desarrollo universal de aplicaciones Windows (**Universal Windows App Development Tools**). Una vez hecho esto, se dispone de diversas plantillas de aplicación, una para cada uno de los lenguajes de programación posibles. Utilizando estas plantillas, se crea un único proyecto que permitirá el desarrollo para todos los tipos de dispositivos deseados.

Las versiones anteriores de Visual Studio solo estaban disponibles para sistema operativo Windows, sin embargo, la aparición de **Visual Studio 2017** ha supuesto que este IDE también se pueda instalar y descargar en dispositivos Mac (aunque las funcionalidades son un poco más reducidas que en Windows). Además, actualmente existen tres versiones distintas de Visual Studio 2017:

- Visual Studio Community 2017: IDE gratuito para ser utilizado por estudiantes, desarrolladores de código abierto y desarrolladores individuales.
- Visual Studio Professional 2017: IDE de pago destinado a pequeños equipos de desarrollo.
- Visual Studio Enterprise 2017: la versión más completa (de pago) para ser utilizada por equipos de mayor tamaño.

2.2.4 Desarrollo nativo multiplataforma

En el año 2014 apareció **Xamarin.Forms**, un framework de desarrollo que actualmente pertenece a **Microsoft**, y que permite crear aplicaciones móviles multiplataforma, generando código que puede ser en gran parte compartido para los desarrollos Android, iOS y Windows.

Con Xamarin, la mayor parte del código se ubica en un proyecto que es compartido por todas las plataformas, el cual contiene la interfaz de usuario y además tiene acceso al framework de .NET, de modo que puede realizar operaciones de entrada/salida, consumir servicios web, manejar ficheros XML y realizar muchas otras funciones. Además de este proyecto con código común, existirán tantos proyectos como plataformas sea necesario soportar. En general, los proyectos específicos de una plataforma serán de reducido tamaño y contendrán pequeños ajustes al código común, así como funcionalidades extra de la plataforma.

Las principales **ventajas** que presenta esta aproximación son:

- Reutilización de código entre plataformas.
- Unificación del lenguaje de programación, que normalmente será C#.
- Unificación del entorno de desarrollo, ya que se utiliza Visual Studio para las tres plataformas.

Sin embargo, Xamarin también presenta algunos **inconvenientes**, como los siguientes:

- A pesar de que la programación se realiza mediante C# en el entorno .NET, siempre será necesario un cierto conocimiento específico de las plataformas para desarrollar algunas de las funcionalidades y/o para mejorar la experiencia de usuario.
- Resulta difícil la integración de librerías complejas que solo están disponibles a través de SDKs de Android e iOS, y para las cuales no se ha desarrollado un paquete NuGet que permita integrar la funcionalidad fácilmente en las aplicaciones Xamarin. Estos SDKs están preparados para ser añadidos de forma sencilla a aplicaciones iOS y Android, pero para ser utilizados en Xamarin requieren de un proceso de transformación que puede resultar complejo, costoso en tiempo y propenso a errores. Esto ocurre por ejemplo si en lugar de utilizar un reproductor de vídeo estándar, se requiere una librería de vídeo específica disponible mediante un Android SDK y un iOS SDK.

En base a estas características, se puede considerar adecuado el **uso de Xamarin** si se cumplen las siguientes condiciones:

- Se necesita desarrollar aplicaciones con buen rendimiento y buena experiencia de usuario, para dos o más plataformas. Esto suele ser un requisito para las aplicaciones de las Administraciones públicas, ya que su objetivo es llegar a todos los ciudadanos, lo que significa que han de ser desarrolladas al menos para Android e iOS.
- Se dispone de personal con conocimientos de .NET y conocimientos generales de desarrollo móvil, o bien personal con conocimientos de desarrollo de aplicaciones móviles Windows, pero no se cuenta con personal especializado en desarrollo iOS y Android.
- La mayor parte de las funcionalidades pueden ser desarrolladas en los proyectos comunes.

Sin embargo, Xamarin **no resulta adecuado** en los siguientes casos:

- Cuando un número importante de funcionalidades requieren implementaciones diferentes según la plataforma. Este es el caso por ejemplo de las notificaciones push o la geolocalización.
- Se requiere la integración de librerías complejas que solo están disponibles a través de SDKs de Android e iOS.

Finalmente, en los siguientes apartados se profundiza en dos de los aspectos más relevantes del desarrollo con Xamarin, como son la arquitectura y los entornos de desarrollo a utilizar.

2.2.4.1 Arquitectura de desarrollo con Xamarin

A continuación se presenta la arquitectura utilizada cuando se desarrollan aplicaciones móviles multiplataforma con Xamarin:

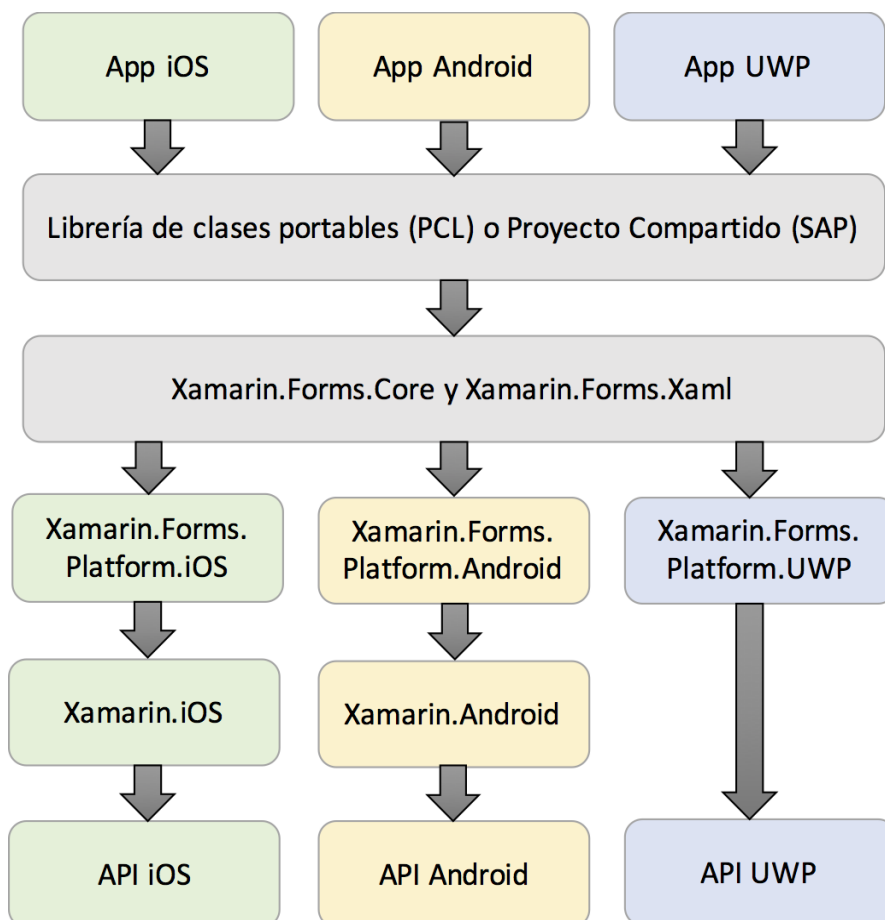


Figura 4. Arquitectura de desarrollo con Xamarin

Como se puede observar en la parte de arriba de la figura anterior, al utilizar Xamarin es posible desarrollar aplicaciones para iOS, Android y la Plataforma Universal de Windows.

Todas estas aplicaciones utilizan un código común que se ubica en librerías de clases portables (PCL: Portable Class Library) o bien en proyectos compartidos (SAP: Shared Asset Project), tal y como se puede ver en la siguiente capa de la figura. Las principales diferencias entre ambas opciones son las siguientes:

- Los **proyectos compartidos o Shared Asset Projects (SAP)** son proyectos que no pueden ser compilados por sí mismos, sino que existen para agrupar código común a incluir en otros proyectos. Sus principales características son:
 - No generan ningún tipo de librería.
 - Se compilan al compilar los proyectos que los referencian, es decir, es como si el código del proyecto compartido se copiara dentro del proyecto que lo referencia, y se compilara como parte de él.
 - No pueden incluir referencias a otros proyectos.
 - Utilizan directivas de compilación que habilitan o deshabilitan secciones de código en función del proyecto que está haciendo uso del proyecto compartido.
 Por ejemplo: `#if __IOS__` para ejecutar una sección de código únicamente en dispositivos iOS, `#if __ANDROID__` si solo se ha de ejecutar en dispositivos Android, `#if __MOBILE__` que incluye tanto iOS como Android, `#if __TVOS__` para televisión etc.

- Las **librerías de clases portables o Portable Class Libraries (PCL)** generan una DLL (Dynamic Link Library) que soporta las plataformas que se hayan especificado. Sus principales características son:
 - Este tipo de proyectos se pueden compilar por sí mismos, y generan una librería (DLL).
 - Utilizan interfaces para añadir funcionalidad extra específica de una plataforma.
 - Es posible referenciar otros proyectos dentro de las PCLs.
 - No utilizan directivas de compilación.

La siguiente capa que se puede observar en la figura está formada por **Xamarin.Forms.Xaml** y **Xamarin.Forms.Core**, dos librerías que implementan el API de **Xamarin.Forms**, y que se describen a continuación:

- **Xamarin.Forms.Xaml** (eXtensible Application Markup Language) permite definir las interfaces de usuario en **Xamarin.Forms**, utilizando lenguaje de marcado en lugar de código. Su uso no es obligatorio a la hora de crear los controles de interfaz, pero puede ser de gran utilidad puesto que resulta más visual que el código, y permite determinar claramente la jerarquía padre/hijo entre los elementos de interfaz. A continuación se muestra un ejemplo de código XAML para representar una etiqueta de texto.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="XamlSamples.HelloXamlPage"
             Title="Hello XAML Page"
             Padding="10, 40, 10, 10">

    <Label Text="Hello, XAML!"
           VerticalOptions="Start"
           HorizontalTextAlignment="Center"
           Rotation="-15"
           IsVisible="true"
           FontSize="Large"
           FontAttributes="Bold"
           TextColor="Aqua" />

</ContentPage>
```

Figura 5. Ejemplo de código XAML

- **Xamarin.Forms.Core** determina la plataforma a la que se destina la aplicación, y en función de ello invoca a la librería **Xamarin.Forms.Platform** correspondiente a esa plataforma.

El siguiente nivel de la arquitectura está formado precisamente por las librerías **Xamarin.Forms.Platform**, existiendo una librería por cada una de las plataformas (iOS, Android y UWP). Estas librerías están formadas por un conjunto de clases que en su mayoría son **renderers** (visualizadores), encargados de transformar los objetos de interfaz **Xamarin.Forms** en controles nativos de la plataforma. Por ejemplo, para cambiar un valor de verdadero a falso o de falso a verdadero, existe un control de tipo *Switch* en **Xamarin.Forms**; sin embargo, este objeto ha de ser transformado en un *UISwitch* en el caso de utilizar iOS, en un *Switch* de Android, o bien en un *ToggleSwitch* de Windows, y esta transformación es la que llevan a cabo las librerías **Xamarin.Forms.Platform**.

En el caso de Android e iOS, por debajo de las librerías **Xamarin.Forms.Platform** se encuentran las librerías **Xamarin.Android** y **Xamarin.iOS**, las cuales permiten crear aplicaciones nativas Android e iOS haciendo uso de los mismos controles de interfaz de usuario que se utilizan en Objective-C y

Java respectivamente, pero aprovechando el potencial que ofrece la librería de clases base (BCL: Base Class Library) de .NET.

2.2.4.2 Entornos de desarrollo

El desarrollo de aplicaciones con Xamarin se lleva a cabo mediante **Visual Studio**, del que ya se ha hablado en la sección de aplicaciones Windows. Actualmente existen versiones de este IDE tanto para Windows como para Mac, aunque este último tiene unas funcionalidades un poco más reducidas que en el caso de Windows.

Además del entorno de desarrollo, para crear aplicaciones Android es necesario instalar también **Xamarin.Android**. En el caso de aplicaciones iOS, se requiere la instalación de **Xamarin.iOS**, y las aplicaciones pueden ser desarrolladas y testeadas tanto en ordenadores Windows como Mac; pero en el caso de desarrollar desde Windows, se necesitará además una conexión a un ordenador Mac con Xamarin.iOS instalado.

2.3 Aplicaciones híbridas

Las aplicaciones híbridas son una combinación de los dos modelos anteriores, y buscan llegar a un compromiso entre facilidad de desarrollo, coste y experiencia de usuario. Este tipo de aplicaciones se desarrollan utilizando tecnologías web como **HTML, CSS y Javascript**, pero a diferencia de las aplicaciones web móviles, **se cargan a través de una WebView**, que no es más que una vista nativa capaz de mostrar contenido web. Esto permite que las aplicaciones híbridas tengan acceso a las funciones del sistema operativo como el acelerómetro o la cámara, normalmente a través de plugins. Además de utilizar tecnologías web, el desarrollo híbrido se suele apoyar en algún tipo de framework Javascript, así como en un framework para el desarrollo híbrido. Todos estos elementos son los que se pueden observar en la siguiente figura, que muestra la arquitectura típica de una aplicación híbrida.

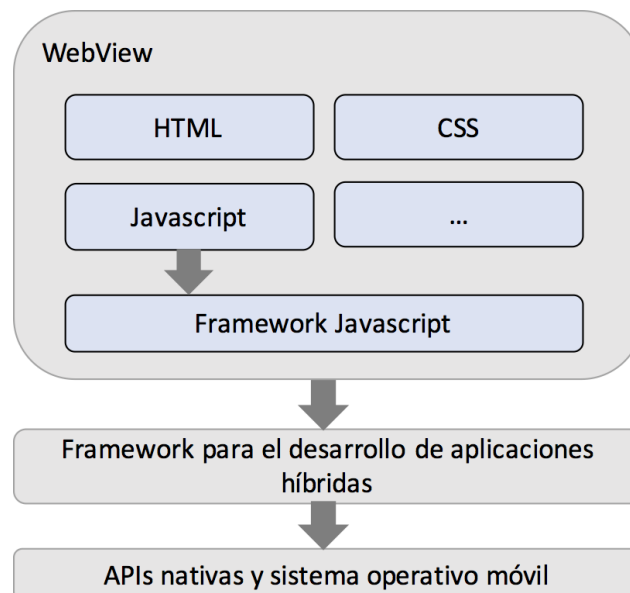


Figura 6. Arquitectura de una aplicación híbrida

En cuanto a su distribución, estas aplicaciones están disponibles a través de las respectivas tiendas, y **requieren descarga e instalación** en el dispositivo.

Por lo tanto, en base a todas estas características, se puede concluir que las principales **ventajas** del desarrollo de aplicaciones híbridas son:

- Facilidad de desarrollo, al utilizar tecnologías web ampliamente conocidas.

- Reutilización de código entre plataformas.
- Acceso a la mayoría de funciones del sistema operativo del dispositivo: cámara, geolocalización, acelerómetro, acceso a ficheros etc.

Sin embargo, el desarrollo híbrido también presenta algunos **inconvenientes**, como los siguientes:

- A pesar de utilizar tecnologías web, siempre será necesario un cierto conocimiento específico de las plataformas para desarrollar algunas de las funcionalidades y/o para mejorar la experiencia de usuario.
- La comunicación de la WebView con el sistema operativo del dispositivo se realiza normalmente mediante plugins, que introducen un nivel más de indirección.
- Algunas características nativas pueden no estar disponibles a través de las WebViews.
- El rendimiento no es tan bueno como en las aplicaciones nativas.
- Requieren un proceso de revisión y aprobación hasta que las aplicaciones están disponibles en las respectivas tiendas: Google Play, App Store etc.
- Requieren instalación.
- Los usuarios han de instalar también las actualizaciones, y éstas requieren de nuevo un proceso de revisión y actualización.

Por este motivo, este tipo de desarrollo se suele utilizar cuando se desea alcanzar un **compromiso entre riqueza de la interfaz de usuario, rendimiento, coste y facilidad de desarrollo**.

2.3.1 Frameworks para el desarrollo de aplicaciones híbridas

Como se ha comentado anteriormente, el desarrollo de este tipo de aplicaciones se suele basar en la utilización de algún framework para el desarrollo de aplicaciones híbridas. A continuación se muestran algunos de estos frameworks:

- **Adobe PhoneGap**: se trata de una distribución de código abierto del framework **Apache Cordova**, y es uno de los frameworks más populares para el desarrollo híbrido. Se basa en la utilización de HTML, CSS y Javascript para construir aplicaciones multiplataforma. Además, mediante el uso de plugins es posible acceder a las APIs nativas que permiten gestionar el acelerómetro, el GPS, la cámara o las notificaciones.
- **Ionic**: está centrado en la parte de interfaz de usuario, y se basa en el uso de componentes, que no son más que controles de interfaz que sirven como base para construir las pantallas de las aplicaciones móviles. Al estar centrado en la parte front-end, el uso de Ionic no excluye la utilización de otros frameworks como por ejemplo PhoneGap. Ionic posee una potente línea de comandos (CLI) y una gran comunidad de desarrolladores. Su principal inconveniente es que no permite seleccionar el framework Javascript, sino que requiere la utilización de AngularJS, por lo son necesarios conocimientos de Angular para poder desarrollar aplicaciones complejas.
- **Onsen UI**: framework de código abierto diseñado para crear aplicaciones híbridas para iOS y Android, que sigue los estándares de diseño de ambas plataformas, y que contiene una gran variedad de componentes de interfaz de usuario diseñados específicamente para dispositivos móviles. Onsen UI está formado por tres capas: componentes CSS, componentes web (escritos en Javascript nativo) y bindings que permiten la integración con frameworks comúnmente utilizados como AngularJS o React.
- **Framework7**: framework de código abierto, muy fácil de utilizar y que permite generar interfaces de usuario con apariencia iOS o Android. Resulta de gran utilidad a la hora de desarrollar prototipos con rapidez. Su principal ventaja es que no requiere un framework Javascript en concreto, sino que se puede utilizar el que se desee. Como inconveniente, destaca la ausencia de herramientas de emulación y empaquetado de aplicaciones móviles.

- **Sencha Ext JS:** framework de pago, que sobresale por su buen rendimiento y por la generación de interfaces que proporcionan muy buena experiencia de usuario. Para ello, se utilizan controles de interfaz predefinidos como calendarios, árboles, formularios o gráficos.

2.4 Conclusiones

El desarrollo de aplicaciones móviles ha crecido exponencialmente en la última década, motivo por el cual los cambios tecnológicos que se producen en este campo son enormes y suceden muy rápidamente. De hecho, a lo largo de estos años han surgido tres aproximaciones al desarrollo de aplicaciones móviles: el desarrollo web móvil, el nativo y el híbrido. Ninguna de estas aproximaciones es mejor que las demás, sino que cada una de ellas tiene sus propias características y presenta sus propias ventajas e inconvenientes, que la hacen más viable en determinados contextos organizativos. Por lo tanto, a la hora de elegir una de las alternativas de desarrollo, resulta fundamental analizar en profundidad las necesidades, características y restricciones de la organización, ya que en función de aspectos como el número de plataformas a soportar, el tiempo de realización del proyecto o el presupuesto máximo disponible, unas soluciones serán mucho más factibles que otras.

3 Distribución de aplicaciones móviles

Tal y como se ha comentado en el capítulo anterior, la puesta a disposición de las aplicaciones móviles se realiza de forma diferente en función del tipo de desarrollo que se haya llevado a cabo. Si se trata de una aplicación web móvil, lo único que se requerirá será un navegador en el dispositivo móvil y una conexión de datos, de modo que se pueda visualizar la aplicación de la misma forma que ocurre con las páginas web que se visitan en los ordenadores de sobremesa.

Sin embargo, si se ha realizado un desarrollo nativo o híbrido, la forma de conseguir acceso a la aplicación es buscarla en las respectivas tiendas de Android, iOS o Windows, para posteriormente descargarla e instalarla en el dispositivo. Esta forma de distribución es la que se describe en este capítulo, en el que se explicará, para cada una de las tres principales plataformas del mercado, cuál es el proceso de empaquetado y subida de la aplicación a la tienda por parte de la organización o administración desarrolladora.

3.1 Distribución de aplicaciones Android

Las aplicaciones Android, una vez desarrolladas y testeadas, **se empaquetan en un archivo con extensión APK**, se firman digitalmente con un certificado, y se publican en la tienda **Google Play**. Para realizar esta publicación, es necesario registrarse con una cuenta de desarrollador de Google Play, por la cual hay que pagar unos 25 euros anuales. Tras el registro, se obtiene acceso a la Play Console, desde la que se gestionan todas las aplicaciones. Para cada nueva aplicación a subir en la tienda, es necesario especificar su nombre, una descripción corta, una descripción completa, los idiomas para los que está disponible y la indicación de si se trata de una aplicación gratuita o de pago. Además, es posible completar la ficha de la aplicación con mucha más información como la categoría a la que pertenece (por ejemplo: educación, comunicaciones...), los datos de contacto para recibir soporte técnico o la política de privacidad, entre otros.

La subida de la aplicación se puede realizar a producción directamente, sin embargo, es común publicar primero la aplicación en modo alfa o beta, de modo que se puedan realizar pruebas antes de realizar la subida definitiva que hará la aplicación disponible a todo el público.

Por otra parte, es necesario tener en cuenta que existen limitaciones de tamaño para los archivos APK a subir, y además estas limitaciones varían en función de la versión de Android que se está utilizando. En concreto, para APKs orientados a Android 2.3 o versiones superiores, el tamaño máximo es de 100 Mega Bytes, mientras que para las versiones anteriores el tamaño máximo es de 50 Mega Bytes. Cabe destacar también que, puesto que los APKs pueden no ser compatibles con todos los dispositivos, es posible subir varios archivos APK para una misma aplicación, pero con diferentes configuraciones de dispositivos.

Otro aspecto importante es el **proceso de revisión** al que tienen que someterse todas las aplicaciones antes de estar disponibles en la tienda. Durante este proceso, se comprueba que la aplicación no infringe las políticas de Google Play, no incluye contenidos inapropiados (como aquellos que pueden fomentar la violencia o el acoso), y cumple con la normativa sobre propiedad intelectual. El tiempo necesario para realizar estas comprobaciones se ha ido reduciendo progresivamente a lo largo del tiempo, y hoy en día es común disponer de la aprobación en unas horas.

Finalmente, la Play Console también permite gestionar las **actualizaciones** de aplicaciones móviles que actualmente ya se encuentran en la tienda. Para ello, se puede optar por dos tipos de publicación:

- Publicación estándar: es el tipo de publicación que se utiliza por defecto, y consiste en procesar y publicar las actualizaciones lo antes posible, normalmente en unas horas.
- Publicación programada: permite controlar el momento exacto en el que se publican las actualizaciones.

3.2 Distribución de aplicaciones iOS

En el caso de iOS, las aplicaciones se empaquetan en un archivo con extensión **IPA**, se firman digitalmente con un certificado, y se publican en la tienda **App Store**. Esta publicación se gestiona a través de la herramienta **iTunes Connect**, y para poder utilizarla es necesario tener una cuenta de usuario, que se obtiene al inscribirse en el programa de desarrolladores Apple (**Apple Developer Program**). El precio anual de este programa ronda los 100 €, aunque también existe el Apple Developer Enterprise Program que cuesta unos 300 € anuales.

En cuanto al proceso a seguir para publicar la aplicación en la App Store, es necesario llevar a cabo los siguientes pasos:

1. Crear un registro de la aplicación en iTunes Connect, el cual contiene toda la información que aparece en la tienda para la aplicación, así como toda la información que se necesita durante el proceso de distribución. Entre esta información se encuentra por ejemplo, el identificador del paquete de aplicación (bundle ID), el nombre de la aplicación, y el tipo de dispositivos a los que se dirige (iOS, tvOS...).
2. Actualizar el número de versión de la aplicación. Este número puede contener hasta tres enteros separados por puntos (por ejemplo 4.5.2), en donde un cambio en el primero de los números significa una revisión o modificación importante, en el segundo una revisión menor, y en el tercero una revisión de mantenimiento.
3. Crear un archivo de la aplicación utilizando la herramienta XCode.
4. Validar el archivo de aplicación, de modo que se pueda comprobar si el proyecto XCode está configurado correctamente para subir la aplicación a la tienda (por ejemplo, puede mostrar un error si no se ha incluido el icono de la aplicación).
5. Subir la aplicación a iTunes Connect.

Al igual que ocurría en Android, puede no ser conveniente subir la aplicación a producción directamente. Por este motivo, se puede utilizar **TestFlight** para publicar primero versiones de prueba, que pueden ser testeadas por usuarios internos (máximo 25) o externos (máximo 2000), según se elija. Los informes de errores proporcionados durante estas pruebas pueden ser muy útiles para depurar problemas y fallos de la aplicación.

En cuanto al **proceso de revisión** que es necesario completar antes de que la aplicación esté disponible al público, se comprueba que la aplicación cumple con las características especificadas en las guías de revisión de la App Store, y que no incluye ningún tipo de contenido ofensivo. Poco a poco este tiempo de revisión se ha ido reduciendo, y actualmente en uno o dos días la revisión ha finalizado.

3.3 Distribución de aplicaciones Windows

Las aplicaciones Windows se empaquetan en archivos con extensión **APPX**, se firman digitalmente con un certificado, y se publican en la **Tienda Windows**. Para ello, es necesario obtener una licencia de desarrollador, la cual permite acceder al centro de desarrollo de Windows (**Windows Dev Center**) desde el que se gestiona la subida de aplicaciones a la tienda. El coste de esta licencia es de unos 100 € para empresas, que se pagan una única vez, ya que no es necesario realizar renovaciones de la licencia.

En cuanto a los pasos a seguir para publicar la aplicación en la tienda, son los siguientes:

1. Reservar un nombre de aplicación en el centro de desarrollo de Windows, tratando de que sea un nombre corto y original, que permita distinguir la aplicación del resto. Además, el

nombre no podrá coincidir con el de otra aplicación. La reserva de nombre dura un año, por lo que si durante ese tiempo no se sube la aplicación a la tienda, el nombre expira.

2. Preparar un envío. Cada envío recoge información sobre una aplicación, como la categoría a la que pertenece (viajes, educación, medicina etc.), la clasificación por edades, la familia o familias de dispositivos en la/s que se puede ejecutar, el precio etc.
3. Crear el paquete de aplicación, normalmente utilizando el asistente de creación de paquetes de Visual Studio.
4. Validar el paquete de aplicación utilizando el Kit para la Certificación de Aplicaciones Windows (Windows App Certification Kit), el cual realiza diversas comprobaciones, como son las pruebas de estabilidad y resistencia, la comprobación de la validez del contenido del manifiesto, pruebas de seguridad, de rendimiento, de integridad del paquete etc.
5. Este proceso de validación forma parte de un proceso de certificación más amplio, que incluye también la comprobación de que la aplicación no incluye software dañino, y que cumple con las políticas de Microsoft. Si se satisfacen todos los requisitos, se produce la certificación de la aplicación, de modo que ésta puede publicarse en la Tienda Windows. El tiempo que puede durar este proceso de revisión/certificación es variable, aunque lo normal es que no exceda de 16 horas.

En cuanto a la publicación de **versiones de prueba de las aplicaciones**, en Windows es posible crear grupos de usuarios pilotos, que han de estar autenticados en la Tienda con una cuenta de Microsoft, y que podrán testear los **paquetes piloto** que se publiquen. Otra opción para realizar pruebas es publicar la aplicación, pero ocultándola a los usuarios en general, y haciéndola disponible solo a determinados evaluadores, que además tendrán que introducir un código promocional para poder acceder a ella y probarla.

Por otra parte, cuando sea necesario **actualizar** aplicaciones que ya están en la tienda, es necesario enviar el nuevo paquete. Existe la posibilidad de lanzar este paquete gradualmente, haciéndolo disponible solo para un determinado porcentaje configurable de clientes. De este modo, se pueden supervisar sus comentarios y los datos analíticos para estar seguro de que la actualización es correcta y no introduce problemas o errores no deseados. Tras examinar los resultados obtenidos con el lanzamiento gradual, es posible finalizar el lanzamiento (si todo ha ido según lo previsto), o bien detenerlo para dejar de distribuir la nueva versión (si ésta da problemas).

3.4 Conclusiones

La forma de distribuir aplicaciones móviles varía en función de la plataforma. Sin embargo, existen similitudes entre los procesos de distribución de Android, iOS y Windows, como son la necesidad de crear un paquete de aplicación, firmarlo con un certificado y enviarlo a la tienda. Además, en todos los casos habrá que verificar que la aplicación cumple las políticas de publicación y que no incluye contenidos inapropiados, lo que significa la superación de un proceso de revisión (de duración variable) como paso previo a la distribución de la aplicación al público. Otra característica importante es la necesidad de probar las aplicaciones para comprobar que no contienen errores graves. La forma en la que cada plataforma permite realizar estas acciones varía, sin embargo, el objetivo final es el mismo en todas ellas, conseguir un lanzamiento exitoso de la aplicación móvil.

4 Tendencias

En este apartado se presentan las principales tendencias relativas a:

- La venta de teléfonos inteligentes y los sistemas operativos móviles más utilizados.
- Las aproximaciones al desarrollo móvil más utilizadas.

Por otra parte, se describen brevemente algunas de las estrategias de programación y plataformas de desarrollo que están cobrando fuerza últimamente. En concreto, se trata del paradigma de desarrollo Mobile First, que cada vez tiene mayor sentido, pues a día de hoy cada vez se accede más a los contenidos a través de dispositivos móviles. Por otra parte, debido al auge que está experimentando el mundo IoT (Internet of Things), se hace referencia a la plataforma Android Things de Google, que está precisamente orientada a desarrollar aplicaciones para este tipo de dispositivos.

4.1 Ventas de teléfonos y sistemas operativos móviles más utilizados

Según Gartner, las ventas de teléfonos inteligentes en el primer trimestre de 2017 (1Q2017) se han incrementado un 9,1% con respecto a las del mismo período del año 2016 (1Q2016), llegando a alcanzar los 380 millones de unidades vendidas a clientes particulares. Actualmente, el fabricante que más dispositivos vende es Samsung, con 78.671.000 unidades vendidas en el primer trimestre de 2017, seguido de Apple (51.992.000 unidades) y Huawei (34.181.000).

En cuanto a sistemas operativos, destacan claramente Android e iOS por encima del resto, aunque el principal líder es Android, que de hecho ha experimentado un 2% de crecimiento, debido en gran parte a la buena aceptación que están teniendo las marcas chinas de teléfonos móviles.

En la siguiente tabla (cuyos datos han sido obtenidos de Gartner), se muestra la evolución de las ventas de teléfonos inteligentes en los últimos trimestres, clasificadas por sistema operativo:

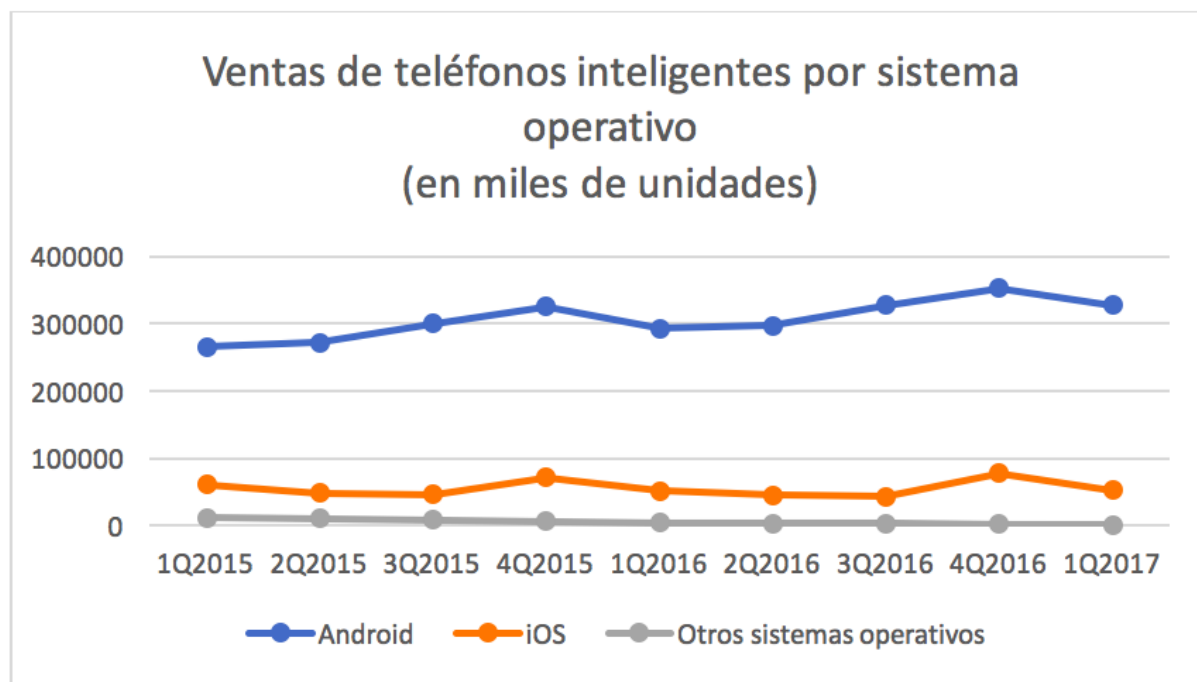


Figura 7. Ventas de teléfonos inteligentes por sistema operativo en los últimos trimestres. Fuente: Gartner

Además, también se muestra un gráfico en el que se puede ver el porcentaje de teléfonos inteligentes por sistema operativo para el primer trimestre del año 2017.



Figura 8. Cuota de mercado por sistema operativo para el primer trimestre de 2017. Fuente: Gartner

4.2 Aproximaciones al desarrollo móvil más utilizadas

En su informe de 2015, Gartner proporcionaba una gráfica sobre las previsiones de uso de los tres tipos de aproximaciones al desarrollo móvil (web, nativa e híbrida), observándose importantes diferencias entre las aplicaciones desarrolladas para el usuario final (aplicaciones de consumo), y las aplicaciones empresariales. Tal y como se puede ver en la siguiente figura, en el caso de aplicaciones dirigidas a usuario final, el 20% de ellas se desarrollan bajo el paradigma web, mientras que un 40% de las aplicaciones son nativas, y otro 40% son híbridas. Estos porcentajes varían notablemente en el caso de aplicaciones empresariales, donde solo el 10% de los desarrollos son nativos, un 30% web, y el 60% restante corresponde a las aplicaciones híbridas.

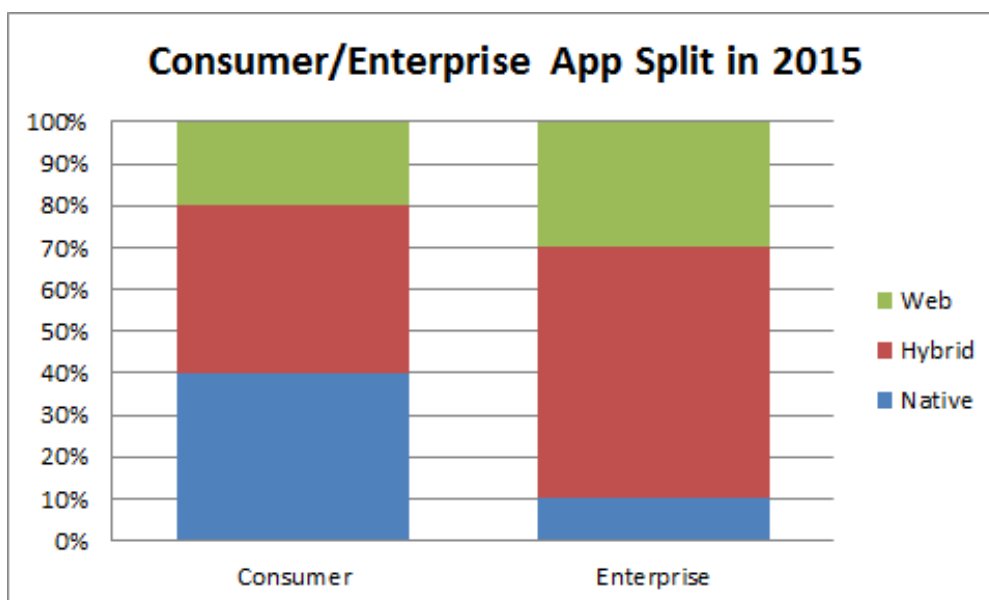


Figura 9. Tendencias en el uso de aproximaciones de desarrollo móvil. Fuente: Gartner

4.3 El paradigma Mobile First

El diseño y desarrollo Mobile First es una vertiente del Responsive Web Design que prioriza la versión móvil de un sitio web, es decir, se comienza diseñando la aplicación para los dispositivos móviles, para posteriormente ir avanzando hacia el diseño para pantallas más grandes mediante una estrategia de mejora progresiva. Las **ventajas** que presenta este enfoque son las siguientes:

- Se concentra en aquellos contenidos y funcionalidades que son realmente necesarios para el usuario.
- No invierte mayor esfuerzo del necesario en la programación (minimiza el código).
- Buen rendimiento en todo tipo de dispositivos (si funciona bien en dispositivos móviles también lo hará en ordenadores de sobremesa o portátiles, que tienen mayores capacidades).
- Acceso rápido a la información.
- Evita el uso de características innecesarias, puesto que algunas de estas características que sí estarían disponibles en los ordenadores de sobremesa y portátiles, no son efectivas o no están disponibles en los dispositivos móviles.
- Elimina el uso de librerías Javascript pesadas.

Esta estrategia debe tener muy en cuenta dos aspectos que son clave en los dispositivos móviles. En primer lugar, éstos tienen un mayor número de **restricciones** que los ordenadores portátiles y de sobremesa, como son las menores capacidades de procesamiento, las pantallas de pequeño tamaño, el acceso desde diferentes localizaciones, las conexiones más lentas y sujetas a mayor número de interrupciones, los usuarios que desean información de forma rápida y fácilmente entendible etc. Pero por otra parte, el acceso desde dispositivos móviles permite también utilizar nuevas **capacidades** mediante las cuales es posible detectar la localización del dispositivo y hacia dónde nos estamos moviendo, utilizar diferentes gestos para interactuar con la pantalla, cambiar la orientación de la interfaz entre horizontal y vertical etc.

4.4 Android Things

Android Things es una plataforma de Google pensada para el Internet de las Cosas (IoT). El desarrollo con Android Things es muy similar al desarrollo Android para dispositivos móviles, y se basa en la utilización de los frameworks, herramientas y APIs Android.

La plataforma extiende el núcleo del framework Android con APIs adicionales proporcionadas por la librería de soporte de Things (Things Support Library), lo cual permite que las aplicaciones se integren con nuevos tipos de hardware.

A continuación se presenta la arquitectura de la plataforma Android Things, en la que se puede observar la gran similitud con la arquitectura de la plataforma Android que fue presentada en el apartado de Android.

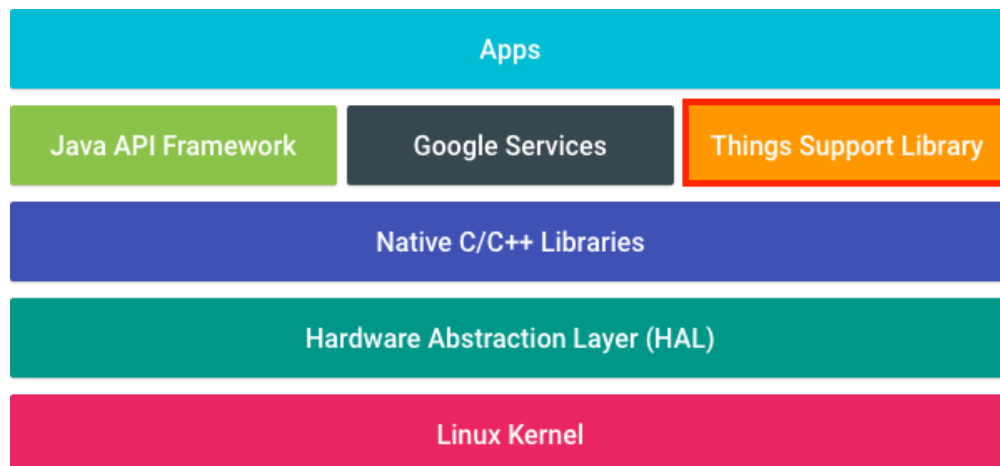


Figura 10. Arquitectura de la plataforma Android Things

4.5 Conclusiones

Android es el principal líder del mercado actualmente (86,10% de cuota de mercado), seguido por iOS (16,68%), mientras que el resto de sistemas operativos han ido perdiendo poder progresivamente, y actualmente solo representan el 0,22% de la cuota de mercado.

En cuanto a aproximaciones de desarrollo, en el caso de aplicaciones dirigidas a usuario final, las principales elecciones son los modelos nativos e híbridos, con un 40% de los desarrollos cada uno; mientras que en el caso de las aplicaciones empresariales el desarrollo nativo solo se utiliza en un 10% de las ocasiones, y el que predomina es el híbrido en un 60% de los casos.

Por último, en lo referente a paradigmas y plataformas que están cobrando cada vez mayor fuerza, destacan la estrategia de desarrollo Mobile First y la plataforma Android Things. Mobile First se centra en comenzar diseñando y desarrollando las aplicaciones web para los dispositivos móviles, para posteriormente ir realizando adaptaciones progresivas a las pantallas de mayor tamaño. En cuanto a Android Things, se trata de una plataforma de Google que permite el desarrollo para dispositivos IoT utilizando los frameworks y herramientas de Android, a los que se han añadido nuevas APIs para poder integrar las aplicaciones con nuevos tipos de hardware.

5 Conclusiones

Las aplicaciones móviles han experimentado un gran crecimiento desde que en el año 2007 apareció el primer iPhone. Además, en 2008 se produjo el despegue de Android, lo que favoreció todavía más ese crecimiento. Actualmente Android e iOS son las dos principales alternativas a la hora de comprar un dispositivo móvil, pues poco a poco BlackBerry y Windows han ido perdiendo peso en el mercado.

Dado que cada vez son más las personas que acceden a todo tipo de aplicaciones a través de sus teléfonos inteligentes, a lo largo de estos años han surgido diferentes aproximaciones para afrontar el desarrollo de aplicaciones móviles. Inicialmente, solo era posible desarrollar **aplicaciones web móviles**, consistentes en aprovechar los conocimientos de desarrollo web, en el que se utilizan tecnologías como HTML, CSS y Javascript, para producir aplicaciones que pueden ser visualizadas no solo en ordenadores de sobremesa sino también en dispositivos móviles, a través del navegador web de dicho dispositivo. Posteriormente aparecieron otras dos opciones: el desarrollo nativo y el híbrido.

El **desarrollo nativo** se centra en desarrollar aplicaciones móviles para una plataforma en concreto (bien sea Android utilizando Android Studio y lenguaje Java, o bien iOS utilizando XCode y lenguaje Objective-C o más recientemente Swift). Este tipo de aplicaciones proporcionan una excelente experiencia de usuario, pero requieren mayor esfuerzo de desarrollo y suponen por lo tanto mayor coste.

Para tratar de aprovechar las ventajas del desarrollo nativo, pero maximizando la capacidad de reutilización de código entre plataformas, surge el **desarrollo nativo multiplataforma**, en el que destaca Xamarin de Microsoft. Sin embargo, esta opción también presenta algunas limitaciones, sobre todo porque no es posible la reutilización de código en el caso de usar determinadas funcionalidades (como notificaciones push o geolocalización), ya que la forma de gestionarlas es diferente según el sistema operativo.

Por último, el desarrollo de **aplicaciones híbridas** busca lograr un equilibrio entre experiencia de usuario, coste y facilidad de desarrollo. El desarrollo híbrido combina características tanto del desarrollo web móvil como del nativo, pues al igual que el primero se basa en la utilización de tecnologías web (HTML, CSS y Javascript), si bien a diferencia de éste las aplicaciones se cargan a través de una vista nativa (WebView) encargada de mostrar contenido web, lo cual permite el acceso a funciones del sistema operativo como el acelerómetro o la cámara, al igual que ocurre con las aplicaciones nativas.

Por lo tanto, dada la diversidad de opciones que existen para desarrollar aplicaciones móviles, es fundamental tener en cuenta las restricciones de la organización y del proyecto que se va a abordar, así como las necesidades de los usuarios, puesto que las características de cada una de estas aproximaciones al desarrollo móvil convierten a unas opciones más viables que otras para un determinado proyecto y contexto.

Por otra parte, hay que tener en cuenta que la **distribución** de la aplicación se realiza de diferente forma en función del tipo de desarrollo que se haya llevado a cabo. Para las aplicaciones web móviles, no se requiere instalación, y por tanto para acceder a ellas basta con un teléfono inteligente con conexión de datos y un navegador web. Sin embargo, para las aplicaciones nativas e híbridas, es necesario crear un paquete con la aplicación, firmarlo, subirlo a la respectiva tienda (App Store, Google Play, Windows Store), y superar un proceso de revisión en el que se comprueba que la aplicación cumple con las políticas de la compañía y no incluye contenido inapropiado.

Por último, cabe destacar que las **Administraciones Públicas** no han sido ajenas a los cambios que ha supuesto el despegue de las aplicaciones móviles, y es por ello que actualmente ya existen numerosas aplicaciones móviles de la Administración, entre las que destacan CI@ve PIN de la Agencia Tributaria, AlertCops de la Policía Nacional y la Guardia Civil (para emitir avisos de seguridad ciudadana), o la aplicación de la Dirección General de Tráfico para conocer las incidencias en las carreteras españolas.

GLOSARIO

- ADT: Android Development Tools
- AOT: Ahead Of Time
- API: Application Programming Interface
- ART: Android Runtime
- BCL: Base Class Library
- CCS: Cascading Style Sheets
- DLL: Dynamic Link Library
- HAL: Hardware Abstraction Layer
- HTML: HyperText Markup Language
- IDE: Integrated Development Environment
- IOT: Internet Of Things
- JIT: Just In Time
- PCL: Portable Class Library
- RWD: Responsive Web Design
- SAP: Shared Asset Project
- SDK: Software Development Kit
- UWP: Universal Windows Platform
- WAP: Wireless Application Protocol
- XAML: eXtensible Application Markup Language

RESUMEN ESQUEMÁTICO

- **Introducción a las aplicaciones móviles:**
 - 2007: Lanzamiento al mercado del primer iPhone.
 - 2008: Inauguración de la App Store + Despegue de Android.
 - Desde entonces: cada vez se venden más teléfonos inteligentes y se desarrollan y usan más aplicaciones móviles.
 - Algunas aplicaciones de las Administraciones Públicas:
 - Cl@ve PIN de la Agencia Tributaria.
 - AlertCops de la Policía Nacional y la Guardia Civil.
- **Características y tecnologías:**
 - Aplicaciones web móviles:
 - Se desarrollan con tecnologías web como HTML, CSS y Javascript.
 - Se cargan a través del navegador web del dispositivo.
 - Muy ligadas al diseño web adaptativo (RWD: Responsive Web Design).
 - Principales ventajas:
 - Menor coste.
 - Facilidad de desarrollo.
 - No requieren instalación.
 - Principales inconvenientes:
 - Peor experiencia de usuario.
 - Peor usabilidad.
 - No permite acceso a funciones del sistema operativo del dispositivo (cámara, geolocalización, acelerómetro etc.)
 - Aplicaciones nativas:
 - Se desarrollan y optimizan para ser utilizadas en un sistema operativo en particular (Android, iOS etc.).
 - Se descargan, instalan y almacenan en el dispositivo.
 - Principales ventajas:
 - Buena usabilidad y riqueza de la interfaz de usuario.
 - Buen rendimiento.
 - Acceso a las funciones del sistema operativo del dispositivo: cámara, geolocalización, acelerómetro, acceso a ficheros etc.
 - Principales inconvenientes:
 - Mayor coste.
 - Se requiere un desarrollo para cada una de las plataformas en las que vaya a utilizarse la aplicación (Android, iOS etc.).
 - *Desarrollo para Android:*
 - Lenguaje: Java.
 - Entorno de desarrollo: Android Studio.
 - Componentes:
 - Actividad.
 - Servicio.
 - Proveedor de contenido.
 - Receptor de mensajes.
 - *Desarrollo para iOS:*
 - Lenguaje: Objective-C o más recientemente Swift.
 - Entorno de desarrollo: XCode.
 - *Desarrollo para Windows:*
 - Plataforma Universal de Windows, que utiliza familias de dispositivos.
 - Entorno de desarrollo: Visual Studio.
 - *Desarrollo multiplataforma:* Xamarin.
 - Reutilización de código entre plataformas, mediante librerías de clases portables y/o proyectos compartidos.
 - Entorno de desarrollo: Visual Studio.

- **Aplicaciones híbridas:**
 - Combinación del desarrollo web móvil y el desarrollo nativo.
 - Compromiso entre riqueza de la interfaz de usuario, rendimiento, coste y facilidad de desarrollo.
 - Se desarrollan con tecnologías web como HTML, CSS y Javascript.
 - Se cargan a través de una WebView (vista nativa capaz de mostrar contenido web).
 - Principales ventajas:
 - Facilidad de desarrollo.
 - Reutilización de código entre plataformas.
 - Acceso a la mayoría de funciones del sistema operativo: cámara, geolocalización...
 - Principales inconvenientes:
 - La comunicación de la WebView con el sistema operativo del dispositivo se realiza normalmente mediante plugins, que introducen un nivel más de indirección.
 - Algunas características pueden no estar disponibles a través de las WebViews.
 - Frameworks:
 - Adobe PhoneGap: basado en Apache Cordova.
 - Ionic.
 - Onsen UI.
 - Framework7.
 - Sencha Ext JS.
- **Distribución de aplicaciones móviles:**
 - Aplicaciones web móviles:
 - Basta con un navegador en el dispositivo móvil y una conexión de datos.
 - Aplicaciones nativas e híbridas:
 - Creación de un paquete + Firma del paquete con un certificado + Envío a la tienda.
 - Proceso de revisión de duración variable: comprobación de que se cumplen las políticas, verificación de que la aplicación no incluye contenidos inapropiados etc.
 - Necesidad de probar la aplicación antes de hacerla disponible a todo el público.
 - *Android:*
 - Extensión del paquete: APK.
 - Tienda: Google Play.
 - *iOS:*
 - Extensión del paquete: IPA.
 - Tienda: App Store.
 - *Windows:*
 - Extensión del paquete: APPX.
 - Tienda: Windows Store.
- **Tendencias:**
 - Sistemas operativos móviles más usados:
 - Android es el líder del mercado, pues según Gartner, en el primer trimestre de 2017, el 86,10% de los teléfonos inteligentes vendidos han sido Android.
 - Le sigue iOS, con un 13,68% de ventas.
 - BlackBerry y Windows han ido perdiendo peso en el mercado, y actualmente solo el 0,22% de los teléfonos vendidos corresponden a sistemas operativos distintos a Android e iOS.
 - Aproximaciones de desarrollo más utilizadas:
 - Para el desarrollo de aplicaciones de usuario final (aplicaciones de consumo):
 - Desarrollo web móvil: 20%.
 - Desarrollo nativo: 40%.
 - Desarrollo híbrido: 40%.
 - Para el desarrollo de aplicaciones empresariales:
 - Desarrollo web móvil: 30%.
 - Desarrollo nativo: 10%.
 - Desarrollo híbrido: 60%.

- Paradigma Mobile First:
 - Vertiente del diseño web adaptativo.
 - Prioriza la versión móvil, y posteriormente se realiza una mejora progresiva para adaptar la aplicación a pantallas de mayor tamaño.
 - Aspectos clave a contemplar:
 - Restricciones de los dispositivos móviles: menor capacidad de procesamiento, pantallas de pequeño tamaño, acceso desde diversas localizaciones, dependencia de la conexión de datos...
 - Nuevas capacidades de los dispositivos móviles: posibilidad de detectar localización y movimiento, gestos para interactuar con la pantalla, cambio de orientación de la pantalla...
- Android Things:
 - Plataforma Google para IoT.
 - Utilización de frameworks y herramientas Android.
 - Nuevas APIs a través de la librería Things Support Library, para poder integrar las aplicaciones con nuevos tipos de hardware.

PREGUNTAS DE TEST

- 1) ¿Cuál de las siguientes no es una aproximación al desarrollo de aplicaciones móviles?
 - a) Desarrollo de aplicaciones nativas.
 - b) Desarrollo de aplicaciones web móviles.
 - c) Desarrollo de aplicaciones híbridas.
 - d) Desarrollo de aplicaciones mixtas.
- 2) ¿Cuál de las siguientes aproximaciones al desarrollo de aplicaciones móviles tiene menor coste?
 - a) Desarrollo de aplicaciones híbridas.
 - b) Desarrollo de aplicaciones nativas.
 - c) Desarrollo multiplataforma de aplicaciones nativas.
 - d) Desarrollo de aplicaciones web móviles.
- 3) Seleccione la respuesta correcta con respecto al desarrollo de aplicaciones móviles nativas:
 - a) La actualización de aplicaciones nativas es más fácil que el de aplicaciones web móviles.
 - b) El desarrollo de aplicaciones nativas requiere profundos conocimientos de tecnologías web.
 - c) El desarrollo de aplicaciones nativas ofrece buen rendimiento y una buena experiencia de usuario.
 - d) Las aplicaciones nativas no requieren instalación en el dispositivo.
- 4) Señale la respuesta incorrecta con respecto al objeto Intent de Android:
 - a) Permite la activación de servicios.
 - b) Permite la activación de proveedores de contenidos.
 - c) Permite la activación de actividades, incluso aunque pertenezcan a otra aplicación.
 - d) Permite la activación de receptores de mensaje.
- 5) ¿Cuál es el nombre del fichero que almacena información de configuración en las aplicaciones iOS?
 - a) Info.plist
 - b) Manifest.plist
 - c) Info.list
 - d) Manifest.list
- 6) ¿Cuál de las siguientes no es una familia de dispositivos de la Plataforma Universal de Windows?
 - a) Familia de dispositivos universal.
 - b) Familia de dispositivos móviles.
 - c) Familia de dispositivos Xbox.
 - d) Todas las anteriores son familias de dispositivos de la Plataforma Universal de Windows.
- 7) ¿Dónde se ubica el código compartido en Xamarin?
 - a) En librerías de clases portables (PCLs: Portable Class Libraries) o proyectos compartidos (SAP: Shared Asset Projects).
 - b) En librerías de clases portables (PCLs: Portable Class Libraries).
 - c) En proyectos compartidos (SAP: Shared Asset Projects).
 - d) En librerías de clases portables (PCLs: Portable Class Libraries) o en proyectos de plataforma.
- 8) ¿Cuál de las siguientes no es una característica de las aplicaciones híbridas?
 - a) Utilizan tecnologías web como HTML, CSS y Javascript.
 - b) Se cargan a través de una WebView, y por lo tanto no requieren instalación.
 - c) Combinan características de las aplicaciones web móviles y de las nativas.
 - d) Tienen acceso a la mayoría de funciones del sistema operativo del dispositivo: cámara, geolocalización, acelerómetro etc.
- 9) ¿Cuál de los siguientes no es un framework para el desarrollo de aplicaciones híbridas?
 - a) Onsen UI.
 - b) Ionic.
 - c) NuGet.
 - d) PhoneGap.
- 10) ¿Qué extensión tienen los paquetes Android?
 - a) IPA.
 - b) APK.
 - c) APP.
 - d) APPX.

- 11) ¿Cuál de las siguientes no es una ventaja del paradigma Mobile First?
- a) Mejora del rendimiento gracias al uso intensivo de librerías Javascript.
 - b) Minimiza la cantidad de código.
 - c) Ayuda a concentrarse en la funcionalidad esencial.
 - d) Evita el uso de características innecesarias.
- 12) ¿Cómo se denomina la plataforma de Google para IoT (Internet of Things)?
- a) Android IoT.
 - b) Android Things.
 - c) Google IoT.
 - d) Google Things.
- 13) Señale la respuesta correcta con respecto a la distribución de aplicaciones móviles:
- a) Las aplicaciones web requieren instalación en el dispositivo.
 - b) Las aplicaciones híbridas requieren instalación en el dispositivo.
 - c) Las aplicaciones nativas requieren instalación en el dispositivo.
 - d) B y C son ciertas.
- 14) ¿Qué es una WebView?
- a) Un control HTML5 para visualizar contenido web.
 - b) Una vista nativa capaz de mostrar contenido web.
 - c) El navegador web de un dispositivo Android.
 - d) Ninguna de las anteriores es correcta.

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	d
2	d
3	c
4	b
5	a
6	d
7	a
8	b
9	c
10	b
11	a
12	b
13	d
14	b

BIBLIOGRAFÍA BÁSICA

1. Android Developers: <https://developer.android.com/index.html>
2. Android Things: <https://developer.android.com/things/index.html>
3. Apple Developer: <https://developer.apple.com/>
4. Gartner Inc.: www.gartner.com
5. Peterson, Clarissa (2014). Learning Responsive Web Design. Publicado por: O'Reilly Media, Inc.
6. Petzold, Charles (2016). Creating Mobile Apps with Xamarin.Forms. Publicado por: Microsoft Press
7. Weyl, Estelle (2013). Mobile HTML5. Publicado por: O'Reilly Media, Inc.
8. Windows Dev Center: <https://developer.microsoft.com/en-us/windows/apps/getstarted>
9. Xamarin: <https://www.xamarin.com/>

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

10. Las aplicaciones móviles de la Administración General del Estado:
<http://administracionelectronica.gob.es/general/aplicaciones/buscadorAplicaciones.htm>